BSc (Hons) in Computer Science & Artificial Intelligence

Course Work: Critical Review and Technical Baseline of a Machine Learning Research Paper

Reviewed Paper:

"FedTGP: Trainable Global Prototypes with Adaptive-Margin-Enhanced Contrastive Learning for Data and Model Heterogeneity in Federated Learning"

Zhang, J., Liu, Y., Hua, Y., & Cao, J. (2024).
Publication: Proceedings of the AAAI Conference on Artificial Intelligence, 38(15), 16768-16776.
DOI: https://doi.org/10.1609/aaai.v38i15.29617

- Student Name : W.A.Thisumi Nimsani
- Student ID : KIC-BSCCSAI-251P-030
- Module : Machine Learning Related Application
- Supervisor / Module Leader : Mr.Ashan Arthnayake
- Date of Submission: Oct 30, 2026

Table of Contents

Overview of the Review **3**

Phase 1. Conceptual mapping and methodological deconstruction **3**

1.1 Problem and research gap 3

1.2 Datasets and data preparation 4

1.3 Methods and architectural choices 5

Phase 2: Baseline implementation and comparison **7**

2.1 Implementation and Comparison Controls 7

2.2 Results and Comparison with the paper 8

Phase 3: Critical review and stress testing. **10**

3.1 Strengths and limitations 10

3.2 Two proposed alternatives 10

3.3 Stress test question and design 11

3.4 Stress-test findings and interpretation 12

Conclusion **13**

References **14**

Appendix A - Implementation Repository **15**

Overview of the Review

This review examines how FedTGP enables clients with different data and different neural networks to learn together. FedTGP and FedProto are implemented on CIFAR-10 and a stress test is run to see how their performance is affected by different client architectures. The results of the experiments favoured FedTGP, although its accuracy gains came at the expense of additional computation.

Phase 1. Conceptual mapping and methodological deconstruction

1.1 Problem and research gap

Federated learning allows organizations or devices to collaborate whilst keeping their data local. FedTGP is concerned with supervised multi-class image classification where the inputs are labelled images and the output is the class of the image. The difficulty of this problem is in two forms of heterogeneity. There is data heterogeneity in that the clients will have different class distributions and different numbers of samples. There is also model heterogeneity in that the clients will be using different network architectures (Zhang et al., 2024).

Conventional FedAvg averages compatible model parameters, which cannot be applied directly to architectures that are incompatible (McMahan et al., 2017). Another option is to share class prototypes: mean feature prototypes are extracted from local images of each class; FedProto then combines these prototypes by averaging them. The class separation can be reduced by using representations from different feature spaces. FedTGP deals with this gap by learning global prototypes that offer more discriminative guidance (Zhang et al., 2024).

1.2 Datasets and data preparation

The paper uses CIFAR-10, CIFAR-100, Oxford Flowers 102 and Tiny-ImageNet, all containing labelled 2D color images. CIFAR-10 and CIFAR-100 each consist of $60,000$ images with

dimensions of $32\ by\ 32$ pixels and three channels, $3,072$ pixel values per image. Their official training sets contain $5,000$ and 500 images per class, respectively (*Krizhevsky et al.*, n.d.). Therefore, CIFAR-100 increases the number of categories while decreasing the number of examples per category.

Flowers102 contains visually similar categories and unequal class sizes across the complete dataset (Nilsback & Zisserman, 2008). Tiny-ImageNet contains 200 classes and 64×64 images (Zhang et al., 2024). Together, the number of categories, the amount of visual detail, and the difficulty of the classification differ.

The fact that the dataset is balanced does not ensure that there is balance at the client level; in the practical scenario described in the paper, Dirichlet is used. With a concentration of 0.1, it permits very unequal distributions among the classes. The pathological setting restricts available categories, with two classes assigned to each CIFAR-10 client (Zhang et al., 2024). Small local samples can produce unreliable prototypes. For independent feature vectors with common covariance Σ, the covariance of their mean is Σ/n, where n is the sample count. This interpretation is based on the assumption of a fixed feature extractor, rather than features that are normally distributed.

The PFLlib pipeline which is being inspected converts images into tensors and applies fixed normalisation (TsingZ0, n.d.-c). Taking 0.5 away from and dividing by 0.5 transforms the interval $[0, 1]$ into $[-1, 1]$, even though it does not guarantee a zero empirical mean or unit variance. Flowers102 generator resizes images to 64×64, potentially losing useful detail; this repository configuration cannot be automatically regarded as equivalent to the published experiments. Image inputs generally do not require tabular missing-value imputation. However, it is necessary to check the file integrity and the labels. Non-stratified local splits may fail to adequately represent rare classes.

1.3 Methods and architectural choices

FedProto shares local class means and aggregates global prototypes (Tan, Long, Liu, et al., 2022). FedMD instead transfers predictions on a shared public dataset (Li & Wang, 2019).

FedPCL employs pretrained representation and prototype-wise contrastive learning (Tan, Long, Ma, et al., 2022). Other alternatives emphasise FedTGP's contribution, which is learning the global prototypes themselves. FedMD and FedPCL were reviewed, not implemented.

The HtFE8 group includes a small CNN, GoogLeNet, MobileNetV2 and ResNet18, ResNet34, ResNet50, ResNet101 and ResNet152. MobileNetV2 uses depthwise convolutions together with inverted residual blocks (Sandler et al., 2018), whereas ResNets use residual connections to support deeper networks (He et al., 2016); their capacities and computational costs are different. A common 512-dimensional feature output enables prototype exchange, although equal dimensions do not guarantee semantic alignment.

Local training combines the use of classification cross-entropy with prototype alignment and the softmax function converts the logits into probabilities and the cross-entropy penalty is applied to the low probability associated with the correct label. Alignment encourages features towards their class's global prototype. The weight has the effect of moderating this influence and demands validation. On the server, trainable class vectors pass through two fully connected layers with ReLU between them, where $ReLU(x) = max(0, x)$ (Zhang et al., 2024).

Adaptive-margin-enhanced contrastive learning uses uploaded local prototypes to train global prototypes. Negative distances serve as class scores. Adding a margin to the matching-class distance makes its comparison harder, encouraging a greater distance advantage over other classes. The adaptive, capped margin limits this requirement. Updated prototypes guide subsequent training and nearest-prototype prediction. Separation is encouraged, not guaranteed for every image (Zhang et al., 2024).

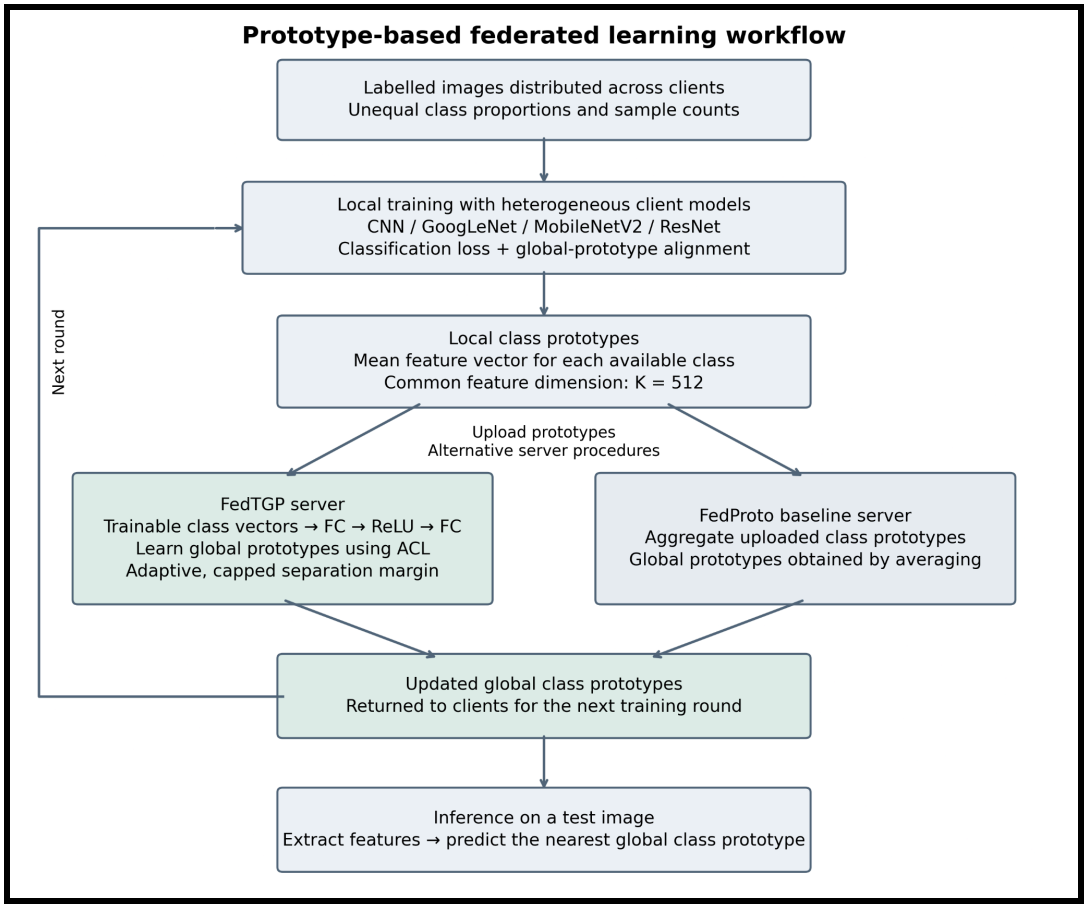


Figure 1. Overall prototype-based learning workflow. Diagram based on Zhang et al. (2024) and Tan, Long, Liu, et al. (2022).

Phase 2: Baseline implementation and comparison

2.1 Implementation and Comparison Controls

FedProto was chosen since prototype sharing offers a relevant baseline for FedTGP. Both implementations made use of the authors' FedTGP framework and HtFLlib components (TsingZ0, n.d.-a, n.d.-b). Code components keep data loading, model building, client training, server updates, and result recording as separate steps. Clients upload class representatives; the server aggregates or learns global prototypes and returns them for further training.The accuracy, the runtime, the settings and the source versions have been saved. The implementation repository is provided in Appendix A.

Google Colab supplied a Tesla T4 GPU with CUDA, and PyTorch took care of the models, the gradients and the optimization. Data processing was supported by NumPy, while CUDA provided acceleration of the computations, even though the clients trained one after another.

Both methods used seed 0, identical saved 20-client partitions, full participation and the same assignment of 8 architectures. Common settings were batch size of 10, one local epoch, learning rate 0.01, feature dimension was 512 and alignment weight $\lambda = 10$. The 100-update budget reflected computational constraints. No systematic search of hyperparameters was conducted. The controls were designed to apply to the two implementations, not to paper's full experimental setup.

Only the practical setting was implemented; the paper evaluated both practical and pathological settings. The generator pooled all $60,000$ CIFAR-10 images before Dirichlet partitioning ($\alpha = 0.1$) and local $75/25$ splitting, producing $44,995$ training and $15,005$ test images. Both methods reused these allocations. Results therefore concern this saved federation rather than the official test split.

The loop evaluated before training: evaluations $0 - 99$ ended after 99 completed updates, although 100 training updates were executed. Therefore, the last recorded accuracy is not a post-update-100 measurement.

Table 1. Phase 2 comparison settings and paper defaults (Zhang et al., 2024).

Setting	Own FedProto and FedTGP	Published default
Data Settings	Practical Only	Practical and Pathological
Clients	20	20
Training budget	100 updates	1000 iterations
Repeated runs	One run per method; seed 0	Three trials
Alignment weight	10	0.1

2.2 Results and Comparison with the paper

FedTGP's best recorded accuracy was 83.46%, compared with FedProto's 71.79%, an advantage of 11.67 percentage points. Last recorded scores were 82.46% and 71.46%; averages over the last ten evaluations were 75.10% and 71.13%, respectively. The smaller sustained gap shows why the best score alone is insufficient.

The published practical accuracies on CIFAR-10 were $88.15 \pm 0.43\%$ for FedTGP and $82.07 \pm 1.64\%$ for FedProto (Zhang et al., 2024); the scores for coursework were respectively 4.69 and 10.28 percentage points lower. These descriptive differences cannot isolate algorithm quality: training budget, repeated trials, alignment weight, the procedures for partitioning and evaluation are different.

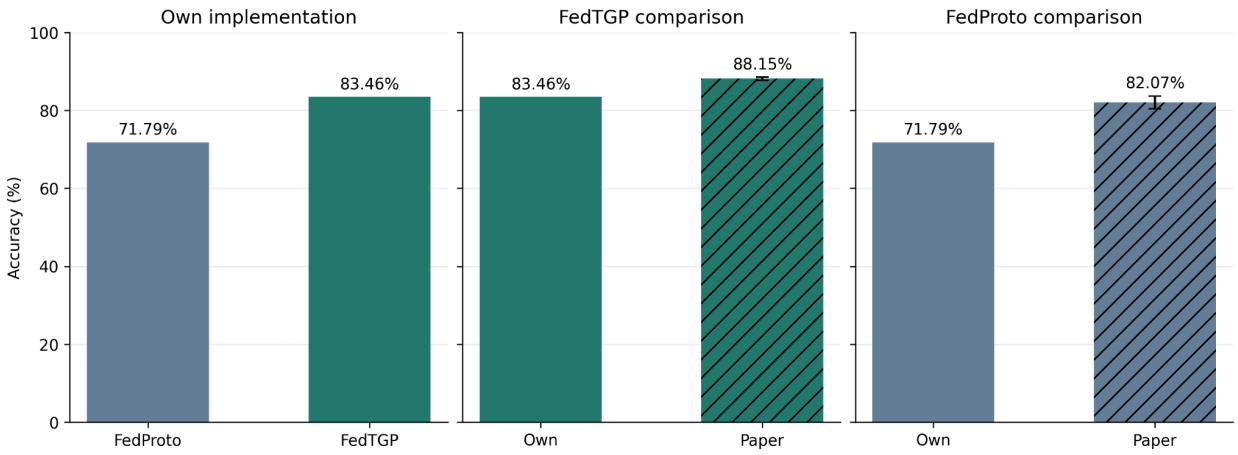


Figure 2. CIFAR-10 accuracy comparisons. Coursework values are seed-0 best recorded scores; published values are three-trial means with standard-deviation error bars (Zhang et al., 2024). Comparisons are contextual, not exact reproductions.

FedTGP's mean logged round time was approximately 189.1 seconds versus 137.8 seconds for FedProto, a 37% increase. Total recorded runtimes were approximately 5.25 and 3.83 hours. Differences in prototype collection also mean this comparison does not isolate the server objective alone.

Phase 3: Critical review and stress testing.

3.1 Strengths and limitations

The paper supports its design through four datasets, two non-IID settings and component ablations (Zhang et al., 2024). However, global prototypes that are separated do not guarantee that useful features will be available on each client. A single representative may fail to show the variation within a class, and small-sample prototypes can be unstable.

Accuracy is suitable for use in multiclass classification, but sample-weighted accuracy tends to favour larger test sets. Mean client accuracy, worst-client accuracy and macro-F1 gives a more comprehensive view. Selecting the best average client-test accuracy during training can produce optimistic estimates (Zhang et al., 2024). This is test-based checkpoint selection, not direct gradient training on test images. A separate validation set would improve evaluation; the same concern applies to Phase 2's best recorded scores.

Round-to-round fluctuations alone do not demonstrate overfitting. Deteriorating independent validation performance alongside improving training performance would provide stronger evidence. Keeping raw images local does not guarantee formal privacy. Communication savings, computation and time-to-target accuracy must be distinguished because slower rounds can still reach a useful score sooner.

3.2 Two proposed alternatives

One alternative uses a few prototypes per class, inspired by mixture prototypes in few-shot learning (Allen et al., 2019). Clients could cluster same-class features and match each uploaded centre to a suitable global prototype. Two or three representatives might preserve appearance differences missed by a single mean. However, alignment and contrastive losses require modification, communication increases and extra prototypes may capture noise. This adaptation was not implemented.

A second alternative replaces the trainable generator with aggregation based on the geometric median for each class. Pillutla et al. (2022) studied robust aggregation of model updates;

adapting it to prototypes might reduce extreme contributions that are achieved by reducing the weighted unsquared distances. The method does not explicitly separate classes or align feature spaces, and iterative computation means efficiency gains require measurement.

Robust aggregation also introduces a fairness concern. A prototype far from the others may reflect a genuinely different client population rather than an unreliable estimate. Reducing its influence could improve average performance while weakening recognition for that minority group. Therefore, the proposed geometric-median alternative should be assessed using individual client results alongside overall accuracy. Comparing its behaviour across clients with different sample sizes would help identify whether robustness is being achieved at the expense of representation. This is a proposed evaluation consideration, not an observed outcome, because neither alternative was implemented or experimentally assessed in the present coursework.

Both proposals require controlled evaluation with fixed partitions, architectures and training budgets, using validation data for configuration selection. Final accuracy, macro-F1, client performance, transmitted bytes and runtime should establish whether benefits justify the costs. Neither proposal was tested in this coursework.

3.3 Stress test question and design

The stress test asks whether FedTGP maintained its advantage as client architecture composition changed. The hypothesis was that the learned global prototypes would be more robust to such changes than the FedProto approach. S1 used five ResNet34 clients. S2 replaced client 4 with a MobileNetV2 and S3 additionally replaced client 3 with a ResNet18. Each configuration was a separate experiment and the models were not swapped during the training process.

CIFAR-10 was partitioned among five clients using a Dirichlet concentration of $\alpha = 0.1$, producing unequal client sample counts and class distributions. The same saved partition, containing $44,999$ training and $15,001$ test examples were reused across both methods, all configurations and all seeds. Three seeds $(0, 1, 2)$, three configurations and two methods produced 18 runs of 100 completed rounds. Stress-test accuracy was measured after 100 completed rounds rather than selecting the highest test score training.

3.4 Stress-test findings and interpretation

FedTGP achieved higher final mean accuracy in all configurations, with gains of 10.91, 15.42 and 12.81 percentage points for S1, S2 and S3, respectively. From S1 to S2, FedProto dropped 5.42 points while FedTGP dropped 0.91. Both improved in S3, showing that additional architecture types did not necessarily reduce accuracy.

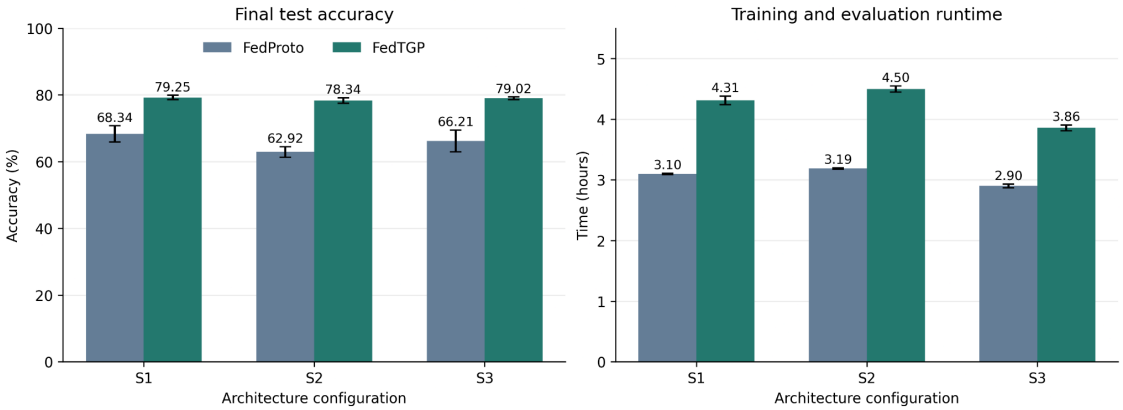


Figure 3. Stress-test accuracy and run time after 100 completed rounds. Bars show means across seeds 0, 1 and 2. Error bars show sample standard deviations. Source: coursework experiments.

FedTGP's accuracy standard deviations ranged from $0.44 - 0.83$ percentage points, versus $1.58 - 3.30$ for FedProto. FedTGP also achieved higher mean worst-client accuracy: $73.63 - 74.85\%$ versus $49.32 - 57.38\%$. These findings support more consistent performance under the tested conditions. Standard deviations describe training variation on one fixed partition, not variation across partitions; the worst-performing client can change between runs.

FedTGP required approximately $33 - 41\%$ more training and evaluation time. Architecture replacements also changed model capacity and suitability for particular clients' data. Client 4 contributed approximately 28.9% of test examples, substantially influencing sample-weighted accuracy. Architectural diversity was therefore not isolated from these factors. Sequential GPU timing, three seeds and a permitted nondeterministic CUDA operation further limit interpretation. Additional partitions and rotating architecture assignments would strengthen

evaluation. Findings support the hypothesis within this experiment, without establishing robustness to untested noise, feature dimensions or every architecture mixture.

Conclusion

The review and experiments suggest that learning global prototypes is useful for heterogeneous image classification. FedTGP performed better than FedProto in the comparisons carried out, even though it involved additional computational cost. The results support its usefulness in these settings, while evaluation limitations and restricted experimental coverage prevents claims of universal superiority.

References

Allen, K., Shelhamer, E., Shin, H., & Tenenbaum, J. (2019). Infinite mixture prototypes for few-shot learning. In K. Chaudhuri & R. Salakhutdinov (Eds.), *Proceedings of the 36th International Conference on Machine Learning* (Vol. 97, pp. 232–241). PMLR. https://proceedings.mlr.press/v97/allen19b.html

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 770–778). https://doi.org/10.1109/CVPR.2016.90

Krizhevsky, A., Nair, V., & Hinton, G. (n.d.). *CIFAR-10 and CIFAR-100 datasets* [Data set]. University of Toronto. https://www.cs.toronto.edu/~kriz/cifar.html

Li, D., & Wang, J. (2019). *FedMD: Heterogenous federated learning via model distillation*. arXiv. https://doi.org/10.48550/arXiv.1910.03581

McMahan, H. B., Moore, E., Ramage, D., Hampson, S., & Agüera y Arcas, B. (2017). Communication-efficient learning of deep networks from decentralized data. In A. Singh & J. Zhu (Eds.), *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics* (Vol. 54, pp. 1273–1282). PMLR. https://proceedings.mlr.press/v54/mcmahan17a.html

Nilsback, M.-E., & Zisserman, A. (2008). Automated flower classification over a large number of classes. In *2008 Sixth Indian Conference on Computer Vision, Graphics & Image Processing* (pp. 722–729). IEEE. https://doi.org/10.1109/ICVGIP.2008.47

Pillutla, K., Kakade, S. M., & Harchaoui, Z. (2022). Robust aggregation for federated learning. *IEEE Transactions on Signal Processing, 70*, 1142–1154. https://doi.org/10.1109/TSP.2022.3153135

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 4510–4520). https://doi.org/10.1109/CVPR.2018.00474

Tan, Y., Long, G., Liu, L., Zhou, T., Lu, Q., Jiang, J., & Zhang, C. (2022). FedProto: Federated prototype learning across heterogeneous clients. *Proceedings of the AAAI Conference on Artificial Intelligence, 36*(8), 8432–8440. https://doi.org/10.1609/aaai.v36i8.20819

Tan, Y., Long, G., Ma, J., Liu, L., Zhou, T., & Jiang, J. (2022). *Federated learning from pre-trained models: A contrastive learning approach*. arXiv. https://doi.org/10.48550/arXiv.2209.10083

TsingZ0. (n.d.-a). *FedTGP* [Computer software]. GitHub. https://github.com/TsingZ0/FedTGP

TsingZ0. (n.d.-b). *HtFLlib* [Computer software]. GitHub. https://github.com/TsingZ0/HtFLlib

TsingZ0. (n.d.-c). *PFLlib* [Computer software]. GitHub. https://github.com/TsingZ0/PFLlib

Zhang, J., Liu, Y., Hua, Y., & Cao, J. (2024). FedTGP: Trainable global prototypes with adaptive-margin-enhanced contrastive learning for data and model heterogeneity in federated learning. *Proceedings of the AAAI Conference on Artificial Intelligence, 38*(15), 16768–16776. https://doi.org/10.1609/aaai.v38i15.29617

Appendix A - Implementation Repository

The implementation notebooks for the phase 2 FedProto-FedTGP comparison and the Phase 3 architecture stress tests are available in the GitHub repository.

Code: https://github.com/ThisumiWijesinghe/FedTGP-Paper-Review